

Deep Learning

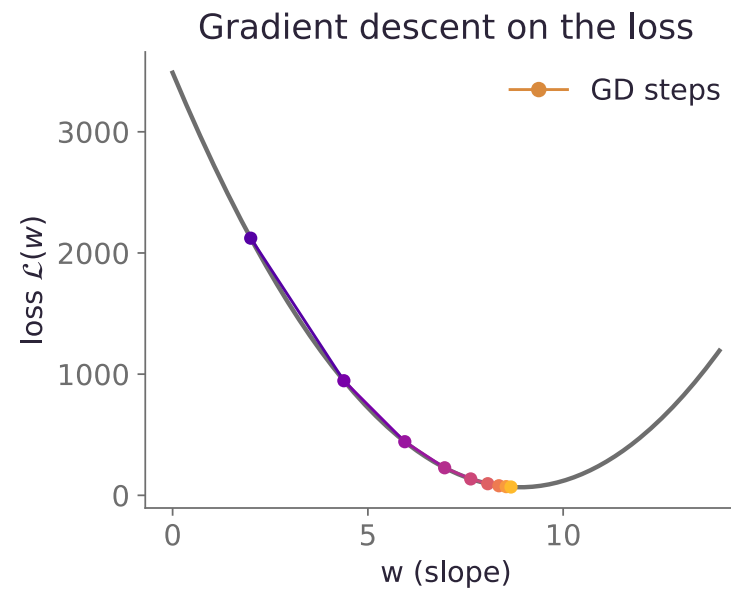
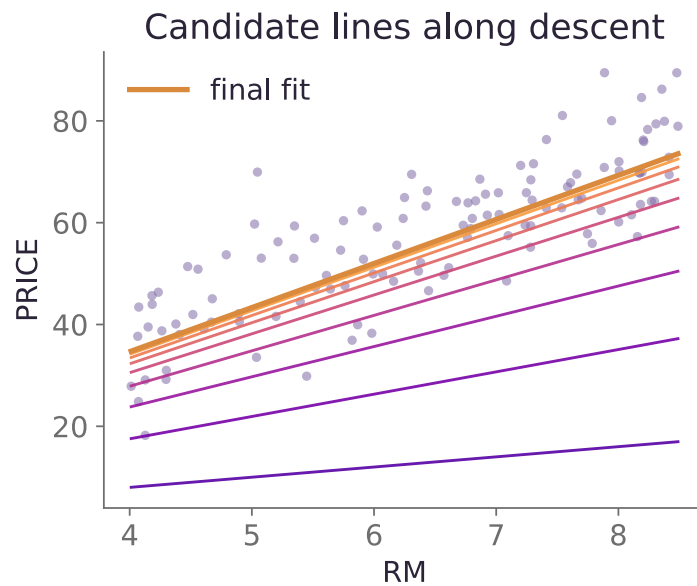
2. Stochastic Gradient Descent

Romain Tavenard

Optimization in general

- **Goal:** Tune model parameters so as to minimize error
- Measuring error
 - Through a cost function / loss function
 - Typical example: Mean Squared Error in regression settings

Gradient descent



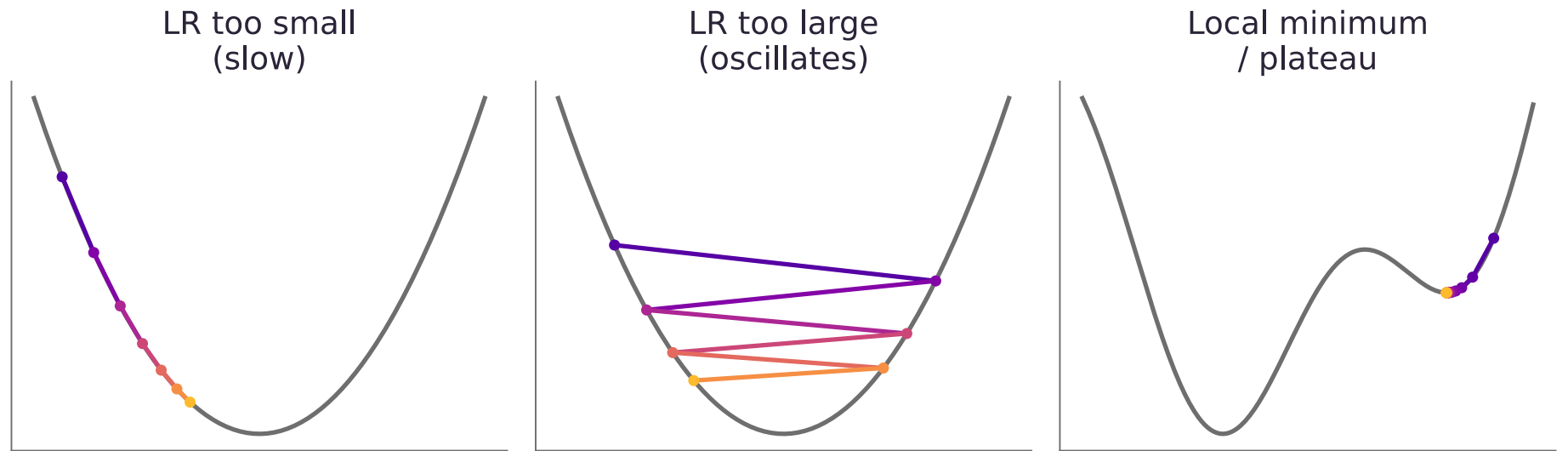
1. Pick a (differentiable) loss function to be minimized e.g.

$$\begin{aligned}\mathcal{L}(w, \{x_i, y_i\}) &= \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(w, x_i, y_i) \\ &= \frac{1}{n} \sum_{i=1}^n (\varphi(w^t x_i) - y_i)^2\end{aligned}$$

2. Use gradient descent

$$w^{(t+1)} \leftarrow w^{(t)} - \rho \nabla_w \mathcal{L}(w^{(t)})$$

Gradient descent in Real Life



Learning-rate pitfalls and local minima

Stochastic Gradient Descent (SGD)

Algorithm 1: Gradient Descent

For each **epoch**:

1. Compute gradient on the **full dataset**:

$$\begin{aligned} G &= \nabla_w \mathcal{L}(w^{(t)}) \\ &= \frac{1}{n} \sum_{i=1}^n \nabla_w \mathcal{L}_i(w^{(t)}) \end{aligned}$$

2. Update weights:

$$w^{(t+1)} \leftarrow w^{(t)} - \rho G$$

Stochastic Gradient Descent (SGD)

Algorithm 1: Gradient Descent

For each **epoch**:

1. Compute gradient on the **full dataset**:

$$\begin{aligned} G &= \nabla_w \mathcal{L}(w^{(t)}) \\ &= \frac{1}{n} \sum_{i=1}^n \nabla_w \mathcal{L}_i(w^{(t)}) \end{aligned}$$

2. Update weights:

$$w^{(t+1)} \leftarrow w^{(t)} - \rho G$$

Algorithm 2: Mini-Batch SGD

For each **epoch**:

1. Split shuffled data into mini-batches of size m
2. For each mini-batch B :
 1. Estimate gradient on the **mini-batch**:

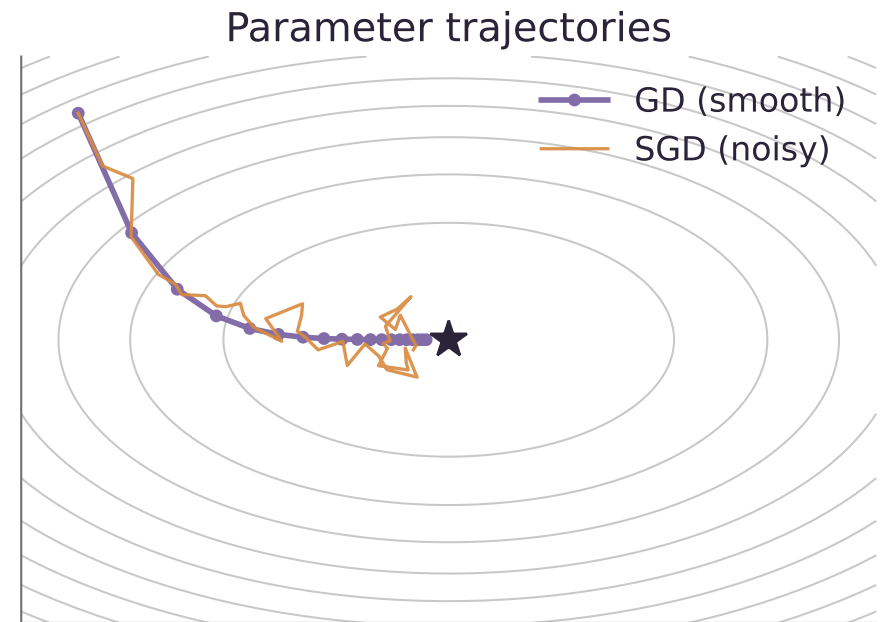
$$G \approx \frac{1}{m} \sum_{i \in B} \nabla_w \mathcal{L}_i(w^{(t)})$$

2. Update weights:

$$w^{(t+1)} \leftarrow w^{(t)} - \rho G$$

Gradient Descent vs Stochastic Gradient Descent

- SGD Cons
 - Subject to high variance
- SGD Pros
 - Faster weight update (each mini-batch)
 - Escape local minima in non-convex settings



*GD: smooth path. SGD: noisy path,
same minimum*

SGD variants: a focus on Adam (1/2)

- **Idea 1**

- ▶ plain SGD: tiny steps on gentle slopes, huge on steep ones
- ▶ want: every parameter advances at a **comparable pace**

- **How:** divide each gradient by **its own magnitude** → size cancels, only **sign** survives:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \rho \frac{\nabla_{\mathbf{w}} \mathcal{L}}{\underbrace{\sqrt{\nabla_{\mathbf{w}} \mathcal{L} \odot \nabla_{\mathbf{w}} \mathcal{L}}}_{\text{sign}(\nabla_{\mathbf{w}} \mathcal{L})}}$$

→ **equal-sized step** in every direction

SGD variants: a focus on Adam (2/2)

- **Idea 2:** add momentum
 - single mini-batch gradient = noisy
 - keep a **running average** → like a ball rolling downhill
[distill]
- **How:** smooth the gradient (and its square) into averages $\mathbf{m}$, $\mathbf{s}$:

$$\begin{aligned}\mathbf{m}^{(t+1)} &= \beta_1 \mathbf{m}^{(t)} + (1 - \beta_1) \nabla_w \mathcal{L} \\ \mathbf{s}^{(t+1)} &= \beta_2 \mathbf{s}^{(t)} + (1 - \beta_2) \nabla_w \mathcal{L} \odot \nabla_w \mathcal{L}\end{aligned}$$

- **Full update** uses smoothed quantities:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \rho \hat{\mathbf{m}} \oslash \sqrt{\hat{\mathbf{s}} + \varepsilon}$$

Conclusion

- Stochastic Gradient Descent
 - Gradient estimates computed on mini-batches
 - More frequent updates
 - Adam is an extremely powerful variant